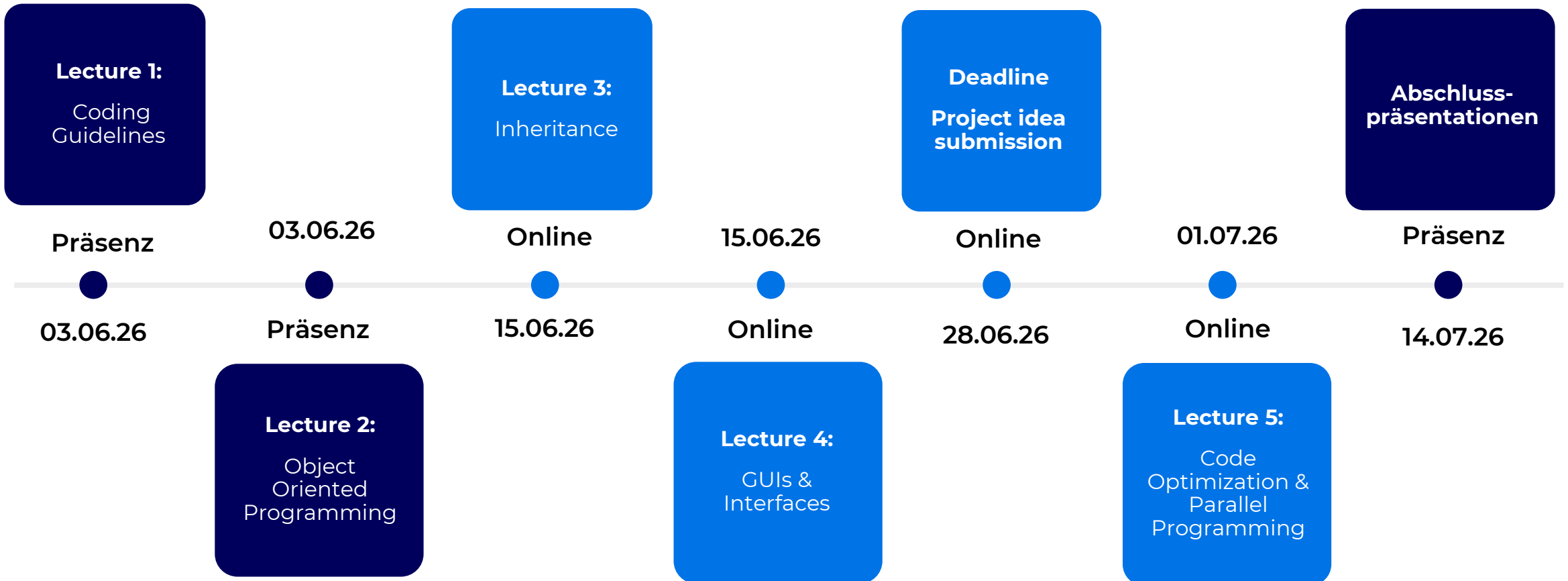


Organisatorisches

Termine



Use Case Ideen

- Bibliothek / Mediathek
 - Verleih physischer/digitaler Medien über mehrere Filialen
 - Jedes Exemplar nur einmal gleichzeitig ausleihbar, Rückgabe filialübergreifend
 - Mahngebühren bei Fristüberschreitung, Vormerkungen
 - Klassen: Medium, Person, Gebäude
Kerninteraktion: `nutzer.ausleihen(medium)` sperrt das Exemplar → zweiter Versuch wirft Exception; `rückgabe()` prüft Frist und aktiviert Vormerkung.
- Krankenhaus / Klinikbetrieb
 - Zuweisung von Betten, Räumen und Geräten an Patienten
 - Bett/Gerät nicht doppelt belegbar, Verlegungen möglich
 - Rechnung bei Entlassung, Notfälle mit Priorität
 - Klassen: Person, Raum, Gerät
Kerninteraktion: `patient.aufnehmen()` belegt ein Bett; `behandeln(gerät)` reserviert Gerät; `entlassen()` gibt frei und rechnet ab.

Use Case Ideen

- Flughafen / Fluggesellschaft
 - Flüge mit Flugzeugen und Gates abwickeln
 - Flugzeug/Gate nur einem Flug zugeordnet, Sitzplatz nach Buchung gesperrt
 - Umbuchung und Entschädigung bei Verspätung/Storno
- Klassen: Fluggerät, Person, Gebäude
Kerninteraktion: `passagier.buchen(sitzplatz)` sperrt den Platz; `flug.zuweisen(flugzeug, gate)` prüft Verfügbarkeit.
- Online-Versandhandel mit Lagerlogistik
 - Bestandsverwaltung in mehreren Lagern, Bestellabwicklung
 - Reservierte Artikel vom Bestand abgezogen (kein Überverkauf)
 - Zustandsprüfung bei Retoure, verschiedene Versandarten
- Klassen: Produkt, Person, Standort
Kerninteraktion: `kunde.bestellen(artikel, menge)` reserviert Bestand → Exception bei Unterdeckung; `retoure()` prüft Zustand und erstattet.

Use Case Ideen

- **Hotelkette**
- Zimmervermietung in mehreren Häusern, plus Zusatzleistungen
- Zimmer pro Zeitraum nur einmal buchbar
- Abrechnung von Übernachtungen + Leistungen beim Auschecken
- Klassen: Raum, Person, Leistung Kerninteraktion: `gast.buchen(zimmer, zeitraum)` sperrt das Zimmer → Exception bei Überlappung; `auschecken()` rechnet ab und setzt „zu reinigen“.
- **Fitnessstudio-Kette**
- Verwaltung von Mitgliedern, Geräten und Kursen über mehrere Standorte
- Kurse mit begrenzten Plätzen, Geräte reservierbar
- Leistungen je nach Mitgliedschaftstyp freigeschaltet
- Klassen: Person, Gerät, Gebäude Kerninteraktion: `mitglied.kursBuchen(kurs)` belegt Platz → Exception wenn voll; `geraetReservieren()` sperrt das Gerät.

Use Case Ideen

- **Essenslieferdienst**
- Bestellungen entgegennehmen, zubereiten, ausliefern
- Fahrer mit begrenzter Kapazität, feste Statusübergänge
- Rabatt bei verspäteter Lieferung
- Klassen: Speise, Person, Fahrzeug Kerninteraktion: kunde.bestellen() erzeugt Bestellung im Statusablauf → Exception bei ungültigem Sprung; fahrer.übernehmen() scheitert bei voller Kapazität.
- **Veranstaltungs- und Konzertmanagement**
- Events an verschiedenen Spielstätten, kategorisierte Tickets
- Sitzplatz nur einmal verkaufbar, Storno bis Frist
- Dynamische Preise, Erstattung bei Absage
- Klassen: Veranstaltung, Person, Gebäude Kerninteraktion: besucher.ticketKaufen(platz) sperrt den Platz → Exception bei Doppelkauf; event.absagen() erstattet alle Tickets.

Use Case Ideen

- Zoo / Tierpark
 - Tiere in Gehegen verwalten, Pflege/Fütterung/Termine planen
 - Gehege mit Kapazität und Art-/Klimaanforderungen, Umsetzen möglich
 - Zufällige Gesundheitsereignisse → Behandlung/Quarantäne
- Klassen: Tier, Person, Gebäude Kerninteraktion: pfleger.einsetzen(tier, gehege) prüft Kapazität/Kompatibilität → Exception; krankheit() triggert Behandlung und ggf. Quarantäne.
- Cloud-Rechenzentrum / IT-Infrastruktur
 - Virtuelle Maschinen auf physischen Servern bereitstellen
 - Server-Ressourcen (CPU/RAM) begrenzt, kein Overbooking, Live-Migration
 - Nutzungsbasierte Abrechnung, Auto-Migration bei Ausfall
- Klassen: Ressource, Person, Gebäude Kerninteraktion: kunde.vmStarten(server) reserviert CPU/RAM → Exception bei Erschöpfung; migrieren(vm, zielServer) verschiebt die Belegung atomar.

Use Cases

- Bitte meldet mir den Use Case per Mail (inkl. der Gruppe, die ihn bearbeitet)
 - Marius.kiskemper@atos.ai
- Keine Themen doppelt → Wer zuerst kommt, bekommt den Use Case

Agenda



1. **Coding Guidelines**
2. **Object Oriented Programming**
3. **Inheritance**
4. **GUIs & Interface**
5. **Code optimization**

01

Classes & Attributes

General overview

Object Oriented Programming

- Using Programmer-defined data types to organize code and data
- Example saving points:
 - variable for x and one for y → using built in type
 - Saving in a list or tuple → using built in type
 - Saving points as own objects → defining new type → class
- Features:
 - Programs with class and method definitions
 - Most computation is expressed by operations on objects
 - Class instances often represents real-world objects and methods their real world behaviour
 - Data and their manipulations are organized into classes and methods

Classes

Creating instances

- `instance = Point()` → instantiation: `instance = object`
- Class names should normally use the CapWords convention
 - Start always with a capital letter
 - Never use the characters 'l' (lowercase letter el), 'O' (uppercase letter oh), or 'I' (uppercase letter eye)
- Classes are used to organize the code
 - Alternative: For example saving personal data in lists or dicts
- Classes allow operations on the created instances
- Multi-line comment as a description

```
class Point:
    """
    description
    """

instance = Point()
```

Attributes

Adding properties to objects

- No need to declare attributes (for example like in Java)
 - Possible to add any attribute to any object at any time of the program
- `instance.x = 3`
- `instance.y = 5`
- Saving values as properties of the object
- Instance attributes are set by the instance and can be different between different instances
- Class attributes are the same for every instance created
 - But can be overwritten of course

Classes with attributes

Example

- Description shows proposed attributes
- Using the previously defined Point class as the value of the corner coordinate
- Vars(box) shows all set attributes as a dictionary
- Attributes can be called and printed like variables

```
class Rectangle:  
    """Represents a rectangle.  
    attributes: width, height, corner.  
    """
```

```
box = Rectangle()  
box.width = 100.0  
box.height = 200.0  
box.corner = Point()  
box.corner.x = 0.0  
box.corner.y = 0.0
```

```
def print_point(p):  
    """Print a Point object in human-readable format."""  
    return'(%g, %g)' % (p.x, p.y)
```

02

Objects

Objects as Return Values

Example

- In general: Objects can be treated like variables
- → Can be passed as a parameter
- Note that here and in the following slides the functions do not yet belong to the classes, but just take the instances as parameters

```
class Rectangle:
    """Represents a rectangle.
    attributes: width, height, corner.
    """

def find_center(rect):
    """Returns a Point at the center of a Rectangle.
    rect: Rectangle
    returns: new Point
    """

    p = Point()
    p.x = rect.corner.x + rect.width/2.0
    p.y = rect.corner.y + rect.height/2.0
    return p
```

```
center = find_center(box)
print('center', end=' ')
print_point(center)
```

Objects are mutable

Example

- Reference to Functional Programming
- Does this mutability of objects rather apply to call by value or call by reference?
- Does it follow functional programming?

→ This approach does not follow the guideline of functional programming

```
def grow_rectangle(rect, dwidth, dheight):  
    """Modifies the Rectangle by adding to its width and height.  
    rect: Rectangle object.  
    dwidth: change in width (can be negative).  
    dheight: change in height (can be negative).  
    """  
  
    rect.width += dwidth  
    rect.height += dheight
```

```
print(box.width)  
print(box.height)  
print('grow')  
grow_rectangle(box, 50, 100)  
print(box.width)  
print(box.height)
```

```
100.0  
200.0  
grow  
150.0  
300.0
```

Copying Objects

Different options

Copy function:

- p1 and p2 contain the same data, but they are not the same Point:
- → both `is` and `==` check for object identity rather than object equivalence

```
p1 = Point()
p1.x = 3
p1.y = 4

p2 = copy.copy(p1)

print_point(p1)
print_point(p2)
print(p1 is p2)
print(p1 == p2)
print(id(p1))
print(id(p2))
```

```
(3, 4)
(3, 4)
False
False
2277810168144
2277808365072
```

Deepcopy function:

- Embedded Point is the same
- Problem as now `grow_rectangle` affects both.
- `Copy.deepcopy()` to copy really everything

```
box2 = copy.copy(box)
print("box2 results:")
print(box2 is box)
print(box2.corner is box.corner)

box3 = copy.deepcopy(box)
print("box3 results:")
print(box3 is box)
print(box3.corner is box.corner)
```

```
box2 results:
False
True
box3 results:
False
False
```

Copying objects

Usage and benefits

- The copied boxes share the same Point object as shown before
- → Invoking a change on the Point object with box1, changes the Point of box2 as well
- → Can lead to a lot of trouble
- That's why functional programming is important
- **Solution:** Deepcopy to remove relationship between the two boxes

```
def shift_center(rect, x, y):  
    """Modifies the Rectangle by setting the center point to the given coordinates.  
    rect: Rectangle object.  
    x: new x position (can be negative).  
    y: new y position (can be negative).  
    """  
  
    rect.corner.x += x  
    rect.corner.y += y
```

```
box2 = copy.copy(box)  
print(print_point(find_center(box)))  
print(print_point(find_center(box2)))  
shift_center(box2, 3, 5)  
print(print_point(find_center(box)))  
print(print_point(find_center(box2)))  
  
print("---")  
  
box3 = copy.deepcopy(box)  
  
print(print_point(find_center(box)))  
print(print_point(find_center(box3)))  
shift_center(box3, 3, 5)  
print(print_point(find_center(box)))  
print(print_point(find_center(box3)))
```

```
(75, 150)  
(75, 150)  
(78, 155)  
(78, 155)  
---  
(78, 155)  
(78, 155)  
(78, 155)  
(81, 160)
```

03

Functional Programming with Classes

Functional Programming Style

Applied with classes

- **Pure functions vs. Modifiers** (functional programming vs. Conventional)
 - Pure functions: has no effect on variables outside the function
 - E.g. applicable by creating a new object out of the given parameters
 - Modifier: modify the parameter objects
- Very easy applicable by using classes
- Pure functions are less error-prone and faster to develop
- Pure functions when reasonable and modifiers only when advantageous
 - → functional programming style

Functional Programming Style

Class Example

- Adding two Time objects together
- Which is a pure function and which is a modifier and why?

```
class Time:
    """Represents the time of day.

    attributes: hour, minute, second
    """

time = Time()
time.hour = 11
time.minute = 53
time.second = 30
```

```
def add_time(t1, t2):
    sum = Time()
    sum.hour = t1.hour + t2.hour
    sum.minute = t1.minute + t2.minute
    sum.second = t1.second + t2.second
    if sum.second >= 60:
        sum.second -= 60
        sum.minute += 1
    if sum.minute >= 60:
        sum.minute -= 60
        sum.hour += 1
    return sum

start = Time()
start.hour = 9
start.minute = 45
start.second = 0

duration = Time()
duration.hour = 1
duration.minute = 35
duration.second = 0

done = add_time(start, duration)
print_time(done)
```

```
def increment(time, seconds):

    time.second += seconds

    while seconds > 60:
        if time.second >= 60:
            time.second -= 60
            time.minute += 1
            seconds -= 60

    while time.minute >= 60:
        if time.minute >= 60:
            time.minute -= 60
            time.hour += 1

    return time
```

04

Methods

Functions / Methods

General Overview

- Method is a function associated with a class and invoked by an instance of a class
- Multi-line comment as a description (functionality, return, parameters)
- Invariants are constraints that parameters have to fulfill
 - For example time parameters have to be positive
 - Seconds and minutes have to be smaller than 61 and so on
 - Write a separate function „validate_parameter“ and call it for every parameter inside the given function → not necessary but a best practice

```
def valid_time(time):  
    """Checks whether a Time object satisfies the invariants.  
    time: Time  
    returns: boolean  
    """  
  
    if time.hour < 0 or time.minute < 0 or time.second < 0:  
        return False  
    if time.minute >= 60 or time.second >= 60:  
        return False  
    return True
```

Methods

Passing vs. calling

- In general exactly the same as a function outside a class
- Novelty: Use self as the first parameter in a class method
 - Reserved word
 - If an object of the class calls the method, the object instance is given to the function as self. If a method takes two objects as parameters, they are called self and other while self is still the object that calls the function
- A method is defined with two parameters (self, param), but is called with only one parameter as the object reference is automatically passed to self

Methods

Passing vs. calling

```
class Time:
    """Represents the time of day.

    attributes: hour, minute, second
    """

def print_time(t):
    """Prints a string representation of the time.
    t: Time object
    """

    print('%02d:%02d:%02d' % (t.hour, t.minute, t.second))

time_instance = Time()
time_instance.hour = 5
time_instance.minute = 40
time_instance.second = 36
print_time(time_instance)
```

Forces two-digit number

```
class Time:
    """Represents the time of day.

    attributes: hour, minute, second
    """

def print_time(self):
    """Prints a string representation of the time.
    t: Time object
    """

    print('%02d:%02d:%02d' % (self.hour, self.minute, self.second))

time_instance = Time()
time_instance.hour = 5
time_instance.minute = 40
time_instance.second = 36
time_instance.print_time()
```

Methods

Arguments

- Positional arguments
 - Mandatory and no default value
 - `send(message, recipient)` or `point(x, y)`
 - Used for the arguments that are fully part of the functions meaning
 - Have natural / intuitive order
 - Keyword arguments
 - Not mandatory and have default values
 - Mostly optional parameters
 - `send(message, main_recipient, cc=None, bcc=None)`
 - Arbitrary argument list
 - Definition: `send(message, *args)` → all arguments after the 2nd are in the tuple `args`
`send(message, **kwargs)` → all following arguments as a dictionary
 - Call: `send('Hello', 'God', 'Mom', 'Cthulhu')`
- Alternative:

Exercise Time

- See Exercise 2:
- [Marius2311/WDSKI25A-Advanced-Programming](#)

05

Special methods

Init Method

Object instantiation

- Init-method (def __init__()) gets invoked when an object of the given class is instantiated
- Much easier to set attributes than assign one value per line
- Instantiation gives opportunity to deliver start values for the attributes
- Parameters are optional, when there are defined default values
- Although attributes can be set at any time, it is best to set all attributes in __init__()
- Not every parameter of init is automatically an attribute of the class

```
class Time:
    """Represents the time of day.

    attributes: hour, minute, second
    """

    def __init__(self, hour=0, minute=0, second=0):
        self.hour = hour
        self.minute = minute
        self.second = second

time = Time()
print("1st instantiation")
time.print_time()

time = Time(9)
print("2nd instantiation")
time.print_time()

time = Time(9,20,54)
print("3rd instantiation")
time.print_time()
```

```
1st instantiation
00:00:00
2nd instantiation
09:00:00
3rd instantiation
09:20:54
```

Str-Method

String representations

- Used to return a string representation of the class
- Allows easy printing of the objects state / attributes
- Can replace methods like print_time()

```
class Time:
    """Represents the time of day.

    attributes: hour, minute, second
    """

    def __str__(self):
        return '%.2d:%.2d:%.2d' % (self.hour, self.minute, self.second)
```

- Without __str__():

```
time = Time(9,20,54)
print(time)
```



```
<__main__.Time object at 0x000001E2C49041D0>
```

- With __str__():

```
time = Time(9,20,54)
print(time)
```



```
09:20:54
```

Other special methods

Operator overloading

- Operator Overloading
- Overriding the usage of certain operators (e.g. in a print-statement)
- Same can be done with other operators
 - - → `__sub__()`
 - * → `__mul__()`
 - ** → `__pow__()`

```
def __add__(self, other):  
    hours = self.hour + other.hour  
    minutes = self.minute + other.minute  
    seconds = self.second + other.second  
    return Time(hours, minutes, seconds)
```

```
time1 = Time(4,30,36)  
time2 = Time(6,13,14)  
print(time1 + time2)
```

10:43:50

Other special methods

Operator overloading

```
class Point:
    """
    description
    """
    def __eq__(self, other):
        return isinstance(other, Point) and self.x == other.x and self.y == other.y
```

- Overloading the equality operator is needed to compare custom built objects → otherwise == behaves like is

Before:

```
p1 = Point()
p1.x = 3
p1.y = 4

p2 = copy.copy(p1)

print_point(p1)
print_point(p2)
print(p1 is p2)
print(p1 == p2)
print(id(p1))
print(id(p2))
```

```
(3, 4)
(3, 4)
False
False
2277810168144
2277808365072
```

After:

```
p1 = Point()
p1.x = 3
p1.y = 4

p2 = copy.copy(p1)

print_point(p1)
print_point(p2)
print(p1 is p2)
print(p1 == p2)
print(id(p1))
print(id(p2))
```

```
(3, 4)
(3, 4)
False
True
2639322978384
2639322978320
```

Polymorphism

Explanation and example

- A function / method is polymorph if it works with multiple input data types
- Designed to count different characters in string
- But works exactly like this also for lists, tuples, ... as inputs
- Improves on type-based dispatch

```
def histogram(input):  
    d = dict()  
  
    for char in input:  
        if char not in d:  
            d[char] = 1  
        else:  
            d[char] += 1  
  
    return d  
  
input1 = "TEST MESSAGE"  
print(f"input type: {type(input1)}")  
print(histogram(input1))  
  
input2 = ["T", "E", "S", "T", " ", "M", "E", "S", "S", "A", "G", "E"]  
print(f"input type: {type(input2)}")  
print(histogram(input2))
```

```
input type: <class 'str'>  
{'T': 2, 'E': 3, 'S': 3, ' ': 1, 'M': 1, 'A': 1, 'G': 1}  
input type: <class 'list'>  
{'T': 2, 'E': 3, 'S': 3, ' ': 1, 'M': 1, 'A': 1, 'G': 1}
```

Main function

Program execution

- Usually the function that executes the whole logic
- Creates the objects
- Calls the methods
- Sequence of steps in the main function should be the execution logic of the program
- If `__name__ == "__main__"` is used to automatically execute the main function
- This is convention for larger projects

```
def main():  
    print("Hello World!")  
  
if __name__ == "__main__":  
    main()
```

Digression

Classes as modules

- Importing classes from other files as modules
- Implementing a whole class and saving it as Class.py
- In another file you can import the class by
 - `import Class` → importing whole class
 - `from Class import func1, func2` → importing specific functions
- Style Guideline:
 1. Import standard libraries (math, random, ...)
 2. Import third party libraries (pandas, numpy, ...)
 3. Import local libraries (Class, Class2, ...)

Exercise Time

Use case: Vehicle rental store

Exercise 1:

- Implement the classes **Car**, **Motorcycle**, **Truck**
- Add an `__init__` and `__str__`-method to each class
- Add to each **typical attributes** (e.g. price, fuel_count, is_clean, license_plate, horse_power, max_speed, ...)
- Also add to each class attributes that are **specific** to this class (cargo_capacity for the truck, has_helmet for the motorcycle, passenger_capacity for the car, ...)
- Add **methods** to each class that every vehicle needs (track_km_driven, fuel_vehicle, ..)
- Add realistic distinguishing methods to the subclasses (e.g. load_truck, add_driver, add_helmet, ...)
- Create different instances for every class and check for consistency
 - Consistency checks: Call methods multiple times and see if the attribute values behave correctly

06

Kahoot

Kahoot

Knowledge Check



07

Summary

Additional Practice Exercises

Classes

Exercise 1:

Modify the given class Bike, by adding these functionalities:

- calculating total_kilometers_moved
- calculating total_miles_moved
- calculating average_speed in kph (km/h)
- import the class Time to calculate and print the total usage time in the format hh:mm:ss
- create the __str__() method and return the output of the total usage time in addition to the total_kilometers_moved
- add the attribute engine power which resembles creating an E-Bike
 - > this should contain the battery-power in Watt
- calculate the Performance of the electric motor in horse power (given the Watt)
<https://de.wikipedia.org/wiki/Pferdestärke>
- create different instances to check for consistency

Additional Practice Exercises

Classes

Exercise 3:

- Create a class Kangaroo
 - Initialize an attribute pouch_contents to an empty list in the __init__()
 - Create a method put_in_pouch which adds an object of any kind to pouch_contents
 - Create a __str__ method that represents the Kangaroo object with it's pouch content
 - Create an object „kanga“ and an object „roo“. Put multiple items in their pouches
 - Put the object roo inside the pouch of kanga
 - Check if appending to pouch is consistent between the two different pouch-lists

Exercise 4:

- Create a class BankAccount with attributes account_number, balance, date_of_opening and customer name
- Create the methods deposit_money, withdraw_money and check_balance
- Add a method that prints the state of all attributes
- Create 5 different instances and execute the different methods

Summary

We learned ...

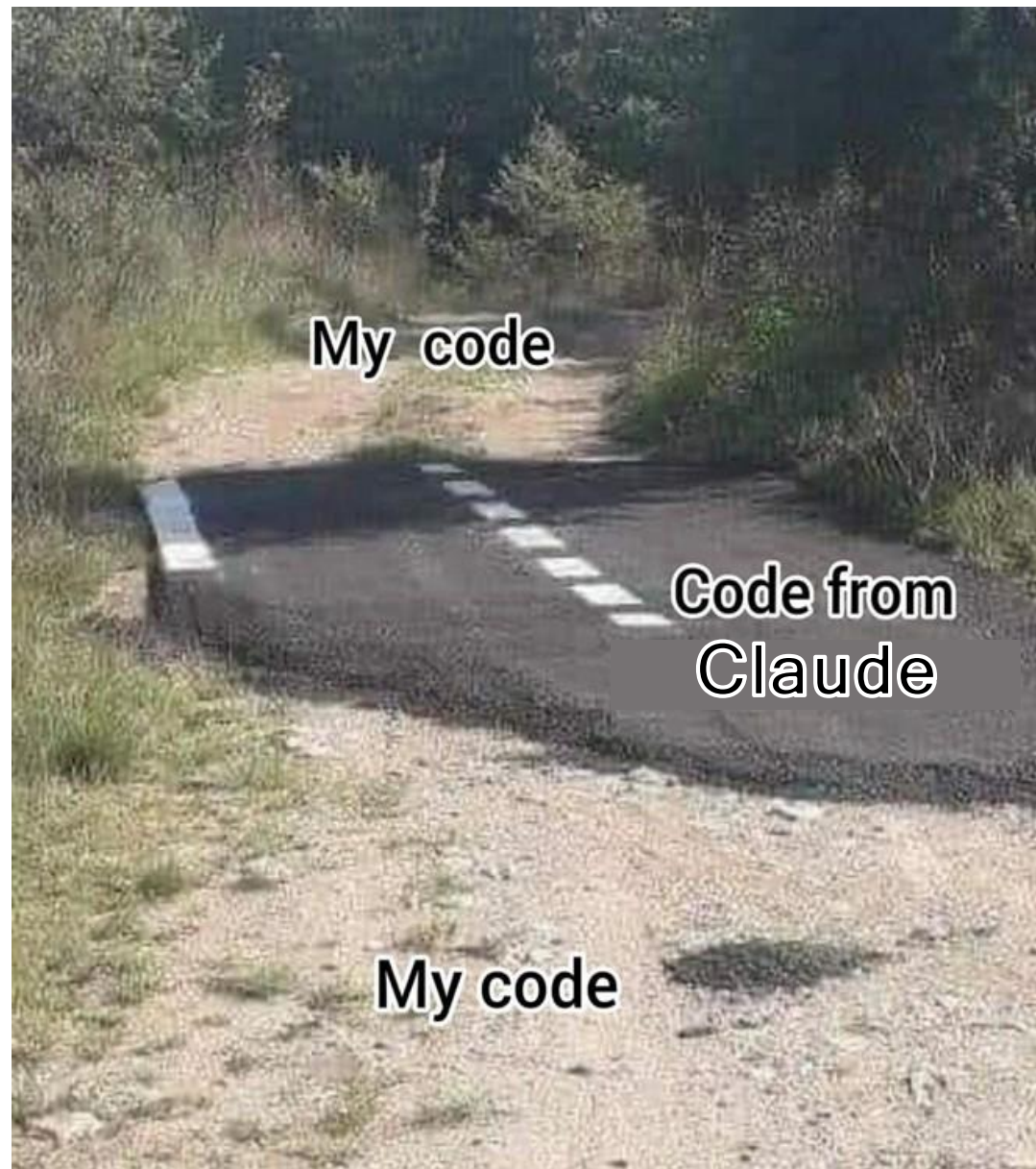
- What classes are
- How to instantiate objects
- What attributes are used for
- How attributes are set with the init-function
- The difference between methods and functions
- How object oriented programming is used to optimize code execution

Summary

Feedback

- Whats your Feedback?
- Changes to the Agenda?
 - Parts towards further topics like Machine Learning Best Practices?
 - GitHub part?
- Short message about project idea

Meme of the day



Next Lecture



- 1. Coding Guidelines**
- 2. Object Oriented Programming**
- 3. Inheritance**
- 4. GUIs & Interface**
- 5. Code optimization**